Cosmos EVM 개발자 도구 설계

개요

이 문서는 Cosmos EVM 개발 및 디버깅을 위한 도구들의 설계를 다룬다. 실제 cosmos/evm 코드베이스의 기존 디버그 인프라를 분석하고, 이를 확장하여 더 강력한 개발자 도구를 제안한다.

분석 대상 코드:

- `x/vm/types/tracer.go` - 트레이서 인터페이스
- `rpc/namespaces/ethereum/debug/api.go` - Debug API
- `rpc/backend/tracing.go` - 트레이싱 백엔드

---

1. 기존 디버그 인프라 분석

1.1 Tracer 시스템 (`x/vm/types/tracer.go`)

cosmos/evm은 go-ethereum의 트레이서 시스템을 통합하여 사용한다:

```
// x/vm/types/tracer.go:16-21
const (
    TracerAccessList = "access_list"
    TracerJSON       = "json"
    TracerStruct     = "struct"
    TracerMarkdown   = "markdown"
)
```

트레이서 생성 함수:

```go
// x/vm/types/tracer.go:25-50
func NewTracer(tracer string, msg core.Message, cfg *params.ChainConfig,
    height int64, timestamp uint64) *tracing.Hooks {

    logCfg := &logger.Config{}

    switch tracer {
    case TracerAccessList:
        blockAddrs := make(map[common.Address]struct{})
        blockAddrs[msg.From] = struct{}{}
        if msg.To != nil {
            blockAddrs[*msg.To] = struct{}{}
        }
        precompiles := vm.ActivePrecompiles(
            cfg.Rules(big.NewInt(height), cfg.MergeNetsplitBlock != nil, timestamp))
        for _, addr := range precompiles {
            blockAddrs[addr] = struct{}{}
        }
        return logger.NewAccessListTracer(msg.AccessList, blockAddrs).Hooks()
    case TracerJSON:
        return logger.NewJSONLogger(logCfg, os.Stderr)
    case TracerMarkdown:
        return logger.NewMarkdownLogger(logCfg, os.Stdout).Hooks()
    case TracerStruct:
        return logger.NewStructLogger(logCfg).Hooks()
    default:
        return NewNoOpTracer()
    }
}
```

트레이서 유형:

트레이서	출력 형식	용도
`access_list`	AccessList	EIP-2930 액세스 리스트 생성
`json`	JSON (stderr)	상세 실행 로그
`struct`	Go struct	프로그래매틱 분석
`markdown`	Markdown (stdout)	문서화용 출력

1.2 NoOpTracer 구조 (`x/vm/types/tracer.go:59-135`)

모든 트레이서 훅의 기본 인터페이스:

```go
// x/vm/types/tracer.go:62-83
func NewNoOpTracer() *tracing.Hooks {
    t := NoOpTracer{}
    return &tracing.Hooks{
        OnTxStart:        t.OnTxStart,
        OnTxEnd:          t.OnTxEnd,
        OnEnter:          t.OnEnter,
        OnExit:           t.OnExit,
        OnOpcode:         t.OnOpcode,
        OnFault:          t.OnFault,
        OnGasChange:      t.OnGasChange,
        OnBlockchainInit: t.OnBlockchainInit,
        OnBlockStart:     t.OnBlockStart,
        OnBlockEnd:       t.OnBlockEnd,
        OnSkippedBlock:   t.OnSkippedBlock,
        OnGenesisBlock:   t.OnGenesisBlock,
        OnBalanceChange:  t.OnBalanceChange,
        OnNonceChange:    t.OnNonceChange,
        OnCodeChange:     t.OnCodeChange,
        OnStorageChange:  t.OnStorageChange,
        OnLog:            t.OnLog,
    }
}
```

훅 이벤트 종류:

카테고리	훅	설명
트랜잭션	`OnTxStart`	트랜잭션 시작
	`OnTxEnd`	트랜잭션 종료
호출	`OnEnter`	함수 진입 (CALL, CREATE 등)
	`OnExit`	함수 반환
실행	`OnOpcode`	각 옵코드 실행
	`OnFault`	실행 오류
가스	`OnGasChange`	가스 변화
상태	`OnBalanceChange`	잔액 변화
	`OnNonceChange`	논스 변화
	`OnCodeChange`	코드 변화
	`OnStorageChange`	스토리지 변화
	`OnLog`	이벤트 로그
블록	`OnBlockStart`	블록 시작
	`OnBlockEnd`	블록 종료

1.3 Debug API (`rpc/namespaces/ethereum/debug/api.go`)

Ethereum 호환 Debug RPC 엔드포인트:

```go
// rpc/namespaces/ethereum/debug/api.go:38-45
type API struct {
    ctx              *server.Context
    logger           log.Logger
    profilingEnabled bool
    backend          backend.EVMBackend
    handler          *HandlerT
}
```

주요 API 메서드:

메서드	기능
`TraceTransaction`	트랜잭션 실행 추적
`TraceBlockByNumber`	블록 내 모든 트랜잭션 추적
`TraceBlockByHash`	블록 해시로 추적
`CpuProfile`	CPU 프로파일링
`MemStats`	메모리 통계
`GcStats`	GC 통계
`BlockProfile`	블록 프로파일링
`MutexProfile`	뮤텍스 프로파일링
`GoTrace`	Go 트레이스
`PrintBlock`	블록 상세 출력

트레이스 구현:

```go
// rpc/namespaces/ethereum/debug/api.go:64-67
func (a *API) TraceTransaction(hash common.Hash,
    config *rpctypes.TraceConfig) (interface{}, error) {
    a.logger.Debug("debug_traceTransaction", "hash", hash)
    return a.backend.TraceTransaction(hash, config)
}
```

1.4 Tracing Backend (`rpc/backend/tracing.go`)

트랜잭션 트레이싱의 실제 구현:

```go
// rpc/backend/tracing.go:23-136
func (b *Backend) TraceTransaction(hash common.Hash,
    config *rpctypes.TraceConfig) (interface{}, error) {
    // 1. 트랜잭션 조회
    transaction, err := b.GetTxByEthHash(hash)
    if err != nil {
        return nil, err
    }

    // 2. 블록 조회
    blk, err := b.CometBlockByNumber(rpctypes.BlockNumber(transaction.Height))
    if err != nil {
        return nil, err
    }

    // 3. 선행 트랜잭션 수집
    var predecessors []*evmtypes.MsgEthereumTx
    for _, txBz := range blk.Block.Txs[:transaction.TxIndex] {
        tx, err := b.ClientCtx.TxConfig.TxDecoder()(txBz)
        if err != nil {
            continue
        }
        for _, msg := range tx.GetMsgs() {
            ethMsg, ok := msg.(*evmtypes.MsgEthereumTx)
            if !ok {
                continue
            }
            predecessors = append(predecessors, ethMsg)
        }
    }

    // 4. TraceTx 쿼리 실행
    traceTxRequest := evmtypes.QueryTraceTxRequest{
        Msg:          ethMessage,
        Predecessors: predecessors,
        BlockNumber:  blk.Block.Height,
        BlockTime:    blk.Block.Time,
        BlockHash:    common.Bytes2Hex(blk.BlockID.Hash),
        // ...
    }

    traceResult, err := b.QueryClient.TraceTx(
        rpctypes.ContextWithHeight(contextHeight), &traceTxRequest)
    if err != nil {
        return nil, err
    }

    return json.Unmarshal(traceResult.Data, &decodedResult)
}
```

---

2. EVM State Inspector 도구

2.1 개요

실시간으로 EVM 상태를 모니터링하고 시각화하는 도구.

```
+-----------------------------------------+
|           EVM State Inspector           |
+-----------------------------------------+
| Block: 12345678  | Tx: 156/200          |
| Time: 2025-01-15 14:30:45 UTC           |
+-----------------------------------------+
|  Accounts (Modified This Block)         |
+-----------------------------------------+
| 0x1234...5678                           |
|   Balance: 100.5 ETH -> 99.3 ETH (-1.2) |
|   Nonce: 42 -> 43                       |
|   Storage Changes: 3                    |
+-----------------------------------------+
| 0xabcd...ef01 (Contract)                |
|   Code Size: 4,256 bytes                |
|   Storage Changes: 15                   |
|   - slot[0x01]: 0x00 -> 0x1a            |
|   - slot[0x02]: 0xff -> 0x00            |
+-----------------------------------------+
```

2.2 핵심 구조

```go
// tools/state-inspector/inspector.go
package stateinspector

import (
    "context"
    "sync"
    "time"

    "github.com/ethereum/go-ethereum/common"
    "github.com/cosmos/evm/x/vm/statedb"
)

// StateInspector provides real-time EVM state monitoring
type StateInspector struct {
    mu           sync.RWMutex
    keeper       *Keeper
    subscribers  map[string]chan<- StateEvent
    eventBuffer  *RingBuffer[StateEvent]

    // Metrics
    metrics      *InspectorMetrics
}

// StateEvent represents a state change event
type StateEvent struct {
    Type        EventType     `json:"type"`
    BlockHeight int64         `json:"blockHeight"`
    TxIndex     uint32        `json:"txIndex"`
    Timestamp   time.Time     `json:"timestamp"`

    // Event-specific data
    Account     *common.Address `json:"account,omitempty"`
    OldValue    interface{}     `json:"oldValue,omitempty"`
    NewValue    interface{}     `json:"newValue,omitempty"`

    // For storage changes
    Slot        *common.Hash    `json:"slot,omitempty"`

    // Gas info
    GasUsed     uint64         `json:"gasUsed,omitempty"`
    GasRemaining uint64        `json:"gasRemaining,omitempty"`
}

type EventType string

const (
    EventBalanceChange EventType = "balance_change"
    EventNonceChange   EventType = "nonce_change"
    EventStorageChange EventType = "storage_change"
    EventCodeChange    EventType = "code_change"
    EventLogEmitted    EventType = "log_emitted"
    EventTxStart       EventType = "tx_start"
    EventTxEnd         EventType = "tx_end"
    EventBlockStart    EventType = "block_start"
    EventBlockEnd      EventType = "block_end"
)
```

2.3 상태 추적 트레이서

```go
// tools/state-inspector/tracer.go
package stateinspector

import (
    "math/big"
    "github.com/ethereum/go-ethereum/common"
    "github.com/ethereum/go-ethereum/core/tracing"
    "github.com/ethereum/go-ethereum/core/types"
)

// InspectorTracer captures state changes for the inspector
type InspectorTracer struct {
    inspector *StateInspector

    // Current context
    blockHeight int64
    txIndex     uint32
    gasStart    uint64
}

// NewInspectorTracer creates a new tracer that feeds the inspector
func NewInspectorTracer(inspector *StateInspector) *tracing.Hooks {
    t := &InspectorTracer{
        inspector: inspector,
    }

    return &tracing.Hooks{
        OnTxStart:       t.OnTxStart,
        OnTxEnd:         t.OnTxEnd,
        OnBalanceChange: t.OnBalanceChange,
        OnNonceChange:   t.OnNonceChange,
        OnStorageChange: t.OnStorageChange,
        OnCodeChange:    t.OnCodeChange,
        OnLog:           t.OnLog,
        OnBlockStart:    t.OnBlockStart,
        OnBlockEnd:      t.OnBlockEnd,
        OnGasChange:     t.OnGasChange,
    }
}

func (t *InspectorTracer) OnBalanceChange(addr common.Address,
    prev, new *big.Int, reason tracing.BalanceChangeReason) {

    t.inspector.emit(StateEvent{
        Type:        EventBalanceChange,
        BlockHeight: t.blockHeight,
        TxIndex:     t.txIndex,
        Account:     &addr,
        OldValue:    prev.String(),
        NewValue:    new.String(),
    })
}

func (t *InspectorTracer) OnStorageChange(addr common.Address,
    slot, prev, new common.Hash) {

    t.inspector.emit(StateEvent{
        Type:        EventStorageChange,
        BlockHeight: t.blockHeight,
        TxIndex:     t.txIndex,
        Account:     &addr,
        Slot:        &slot,
        OldValue:    prev.Hex(),
        NewValue:    new.Hex(),
    })
}

func (t *InspectorTracer) OnLog(log *types.Log) {
    t.inspector.emit(StateEvent{
        Type:        EventLogEmitted,
        BlockHeight: t.blockHeight,
        TxIndex:     t.txIndex,
        Account:     &log.Address,
        NewValue:    log,
```

```
    })
}
```

2.4 WebSocket 실시간 스트리밍

```go
// tools/state-inspector/websocket.go
package stateinspector

import (
    "encoding/json"
    "net/http"

    "github.com/gorilla/websocket"
)

var upgrader = websocket.Upgrader{
    ReadBufferSize:  1024,
    WriteBufferSize: 1024,
    CheckOrigin:     func(r *http.Request) bool { return true },
}

// WebSocketHandler handles WebSocket connections for real-time updates
func (s *StateInspector) WebSocketHandler(w http.ResponseWriter,
    r *http.Request) {

    conn, err := upgrader.Upgrade(w, r, nil)
    if err != nil {
        http.Error(w, err.Error(), http.StatusBadRequest)
        return
    }
    defer conn.Close()

    // Parse filter from query params
    filter := parseFilter(r.URL.Query())

    // Create subscription
    subID := generateSubscriptionID()
    events := make(chan StateEvent, 100)
    s.subscribe(subID, events)
    defer s.unsubscribe(subID)

    // Stream events
    for event := range events {
        if !filter.Match(event) {
            continue
        }

        data, err := json.Marshal(event)
        if err != nil {
            continue
        }

        if err := conn.WriteMessage(websocket.TextMessage, data); err != nil {
            break
        }
    }
}

// EventFilter defines criteria for filtering events
type EventFilter struct {
    EventTypes []EventType      `json:"eventTypes,omitempty"`
    Accounts   []common.Address `json:"accounts,omitempty"`
    MinBlock   int64            `json:"minBlock,omitempty"`
    MaxBlock   int64            `json:"maxBlock,omitempty"`
}

func (f *EventFilter) Match(event StateEvent) bool {
    // Type filter
    if len(f.EventTypes) > 0 {
        found := false
        for _, t := range f.EventTypes {
            if t == event.Type {
                found = true
                break
            }
        }
        if !found {
            return false
        }
    }
```

```go
    // Account filter
    if len(f.Accounts) > 0 && event.Account != nil {
        found := false
        for _, addr := range f.Accounts {
            if addr == *event.Account {
                found = true
                break
            }
        }
        if !found {
            return false
        }
    }

    // Block range filter
    if f.MinBlock > 0 && event.BlockHeight < f.MinBlock {
        return false
    }
    if f.MaxBlock > 0 && event.BlockHeight > f.MaxBlock {
        return false
    }

    return true
}
```

2.5 State Diff 시각화

```go
// tools/state-inspector/diff.go
package stateinspector

import (
    "fmt"
    "strings"

    "github.com/ethereum/go-ethereum/common"
)

// StateDiff represents the difference between two states
type StateDiff struct {
    BlockHeight int64                          `json:"blockHeight"`
    Accounts    map[common.Address]*AccountDiff `json:"accounts"`

    // Summary
    TotalBalanceChange *big.Int `json:"totalBalanceChange"`
    StorageChanges     int      `json:"storageChanges"`
    NewContracts       int      `json:"newContracts"`
}

type AccountDiff struct {
    Address       common.Address            `json:"address"`
    BalanceDiff   *BalanceDiff              `json:"balance,omitempty"`
    NonceDiff     *NonceDiff                `json:"nonce,omitempty"`
    CodeDiff      *CodeDiff                 `json:"code,omitempty"`
    StorageDiff   map[common.Hash]*SlotDiff `json:"storage,omitempty"`

    IsNew         bool `json:"isNew"`
    IsDestroyed   bool `json:"isDestroyed"`
}

type BalanceDiff struct {
    Before *big.Int `json:"before"`
    After  *big.Int `json:"after"`
    Delta  *big.Int `json:"delta"`
}

type NonceDiff struct {
    Before uint64 `json:"before"`
    After  uint64 `json:"after"`
}

type CodeDiff struct {
    BeforeHash common.Hash `json:"beforeHash"`
    AfterHash  common.Hash `json:"afterHash"`
    SizeDelta  int         `json:"sizeDelta"`
}

type SlotDiff struct {
    Key    common.Hash `json:"key"`
    Before common.Hash `json:"before"`
    After  common.Hash `json:"after"`
}

// FormatDiff formats the diff for terminal output
func (d *StateDiff) FormatTerminal() string {
    var sb strings.Builder

    sb.WriteString(fmt.Sprintf("Block #%d State Diff\n", d.BlockHeight))
    sb.WriteString(strings.Repeat("=", 60) + "\n\n")

    for addr, acct := range d.Accounts {
        if acct.IsNew {
            sb.WriteString(fmt.Sprintf("[NEW] %s\n", addr.Hex()))
        } else if acct.IsDestroyed {
            sb.WriteString(fmt.Sprintf("[DESTROYED] %s\n", addr.Hex()))
        } else {
            sb.WriteString(fmt.Sprintf("%s\n", addr.Hex()))
        }

        if acct.BalanceDiff != nil {
            delta := acct.BalanceDiff.Delta
            sign := "+"
            if delta.Sign() < 0 {
                sign = ""
```

```go
            }
            sb.WriteString(fmt.Sprintf("  Balance: %s → %s (%s%s)\n",
                formatEther(acct.BalanceDiff.Before),
                formatEther(acct.BalanceDiff.After),
                sign,
                formatEther(delta)))
        }

        if acct.NonceDiff != nil {
            sb.WriteString(fmt.Sprintf("  Nonce: %d → %d\n",
                acct.NonceDiff.Before, acct.NonceDiff.After))
        }

        if len(acct.StorageDiff) > 0 {
            sb.WriteString(fmt.Sprintf("  Storage (%d changes):\n",
                len(acct.StorageDiff)))
            for slot, diff := range acct.StorageDiff {
                sb.WriteString(fmt.Sprintf("    [%s]: %s → %s\n",
                    slot.Hex()[:10]+"...",
                    diff.Before.Hex()[:10]+"...",
                    diff.After.Hex()[:10]+"..."))
            }
        }

        sb.WriteString("\n")
    }

    return sb.String()
}
```

---

3. Consensus Visualizer 도구

3.1 개요

CometBFT 합의 프로세스를 실시간으로 시각화하는 도구.

```
+-------------------------------------------------------------+
|                    Consensus Visualizer                      |
+-------------------------------------------------------------+
| Height: 12345678  Round: 0  Step: Prevote                    |
| Proposer: validator1 (cosmos1abc...)                         |
+-------------------------------------------------------------+
|                     Validator Votes                          |
+-------------------------------------------------------------+
| Validator        | Voting Power | Prevote | Precommit        |
|------------------|--------------|---------|------------------|
| validator1       | 25%          |   [x]   |   [ ]            |
| validator2       | 20%          |   [x]   |   [ ]            |
| validator3       | 18%          |   [x]   |   [ ]            |
| validator4       | 15%          |   [ ]   |   [ ]            |
| validator5       | 12%          |   [x]   |   [ ]            |
| validator6       | 10%          |   [x]   |   [ ]            |
+-------------------------------------------------------------+
| Prevote: 85% (5/6)  |  Precommit: 0% (0/6)                   |
| +2/3 Threshold: 67% |  Status: Waiting for precommit         |
+-------------------------------------------------------------+
```

3.2 핵심 구조

```go
// tools/consensus-viz/visualizer.go
package consensusviz

import (
    "sync"
    "time"

    "github.com/cometbft/cometbft/types"
)

// ConsensusVisualizer tracks and visualizes consensus state
type ConsensusVisualizer struct {
    mu          sync.RWMutex

    // Current state
    height      int64
    round       int32
    step        RoundStepType
    proposer    types.Address

    // Validators
    validators  *types.ValidatorSet

    // Votes
    prevotes    *types.VoteSet
    precommits  *types.VoteSet

    // History
    roundHistory []*RoundInfo

    // Subscribers
    subscribers  map[string]chan<- ConsensusEvent
}

type RoundStepType int

const (
    RoundStepNewHeight     RoundStepType = iota
    RoundStepNewRound
    RoundStepPropose
    RoundStepPrevote
    RoundStepPrevoteWait
    RoundStepPrecommit
    RoundStepPrecommitWait
    RoundStepCommit
)

type RoundInfo struct {
    Height       int64         `json:"height"`
    Round        int32         `json:"round"`
    Proposer     types.Address `json:"proposer"`
    StartTime    time.Time     `json:"startTime"`
    EndTime      time.Time     `json:"endTime"`
    Duration     time.Duration `json:"duration"`

    // Vote stats
    PrevoteCount    int     `json:"prevoteCount"`
    PrevotePower    int64   `json:"prevotePower"`
    PrecommitCount  int     `json:"precommitCount"`
    PrecommitPower  int64   `json:"precommitPower"`

    // Result
    Committed    bool          `json:"committed"`
    BlockHash    []byte        `json:"blockHash,omitempty"`
}

type ConsensusEvent struct {
    Type      EventType     `json:"type"`
    Timestamp time.Time     `json:"timestamp"`
    Height    int64         `json:"height"`
    Round     int32         `json:"round"`

    // Event-specific
    Vote      *types.Vote   `json:"vote,omitempty"`
    Proposal  *types.Proposal `json:"proposal,omitempty"`
    Block     *types.Block  `json:"block,omitempty"`
```

```go
}

type EventType string

const (
    EventNewRound    EventType = "new_round"
    EventProposal    EventType = "proposal"
    EventPrevote     EventType = "prevote"
    EventPrecommit   EventType = "precommit"
    EventCommit      EventType = "commit"
    EventTimeout     EventType = "timeout"
)
```

3.3 투표 추적

```go
// tools/consensus-viz/votes.go
package consensusviz

import (
    "github.com/cometbft/cometbft/types"
)

// VoteTracker tracks voting progress
type VoteTracker struct {
    validatorSet *types.ValidatorSet
    totalPower   int64

    prevotes    map[string]*types.Vote // validator address -> vote
    precommits  map[string]*types.Vote
}

func NewVoteTracker(valSet *types.ValidatorSet) *VoteTracker {
    return &VoteTracker{
        validatorSet: valSet,
        totalPower:   valSet.TotalVotingPower(),
        prevotes:     make(map[string]*types.Vote),
        precommits:   make(map[string]*types.Vote),
    }
}

func (vt *VoteTracker) AddVote(vote *types.Vote) {
    addrStr := vote.ValidatorAddress.String()

    switch vote.Type {
    case types.PrevoteType:
        vt.prevotes[addrStr] = vote
    case types.PrecommitType:
        vt.precommits[addrStr] = vote
    }
}

// GetPrevoteProgress returns prevote stats
func (vt *VoteTracker) GetPrevoteProgress() (count int, power int64, percentage float64) {
    count = len(vt.prevotes)
    for addrStr := range vt.prevotes {
        _, val := vt.validatorSet.GetByAddress([]byte(addrStr))
        if val != nil {
            power += val.VotingPower
        }
    }
    percentage = float64(power) / float64(vt.totalPower) * 100
    return
}

// GetPrecommitProgress returns precommit stats
func (vt *VoteTracker) GetPrecommitProgress() (count int, power int64, percentage float64) {
    count = len(vt.precommits)
    for addrStr := range vt.precommits {
        _, val := vt.validatorSet.GetByAddress([]byte(addrStr))
        if val != nil {
            power += val.VotingPower
        }
    }
    percentage = float64(power) / float64(vt.totalPower) * 100
    return
}

// HasQuorum checks if 2/3+ voting power has been reached
func (vt *VoteTracker) HasQuorum(voteType types.SignedMsgType) bool {
    var power int64
    var votes map[string]*types.Vote

    switch voteType {
    case types.PrevoteType:
        votes = vt.prevotes
    case types.PrecommitType:
        votes = vt.precommits
    }

    for addrStr := range votes {
        _, val := vt.validatorSet.GetByAddress([]byte(addrStr))
```

```go
        if val != nil {
            power += val.VotingPower
        }
    }

    // 2/3+ majority
    return power > vt.totalPower*2/3
}

// GetValidatorVoteStatus returns vote status for each validator
func (vt *VoteTracker) GetValidatorVoteStatus() []ValidatorVoteStatus {
    result := make([]ValidatorVoteStatus, 0, vt.validatorSet.Size())

    for idx := 0; idx < vt.validatorSet.Size(); idx++ {
        _, val := vt.validatorSet.GetByIndex(int32(idx))
        addrStr := val.Address.String()

        status := ValidatorVoteStatus{
            Address:     val.Address,
            VotingPower: val.VotingPower,
            PowerPercent: float64(val.VotingPower) / float64(vt.totalPower) * 100,
        }

        if vote, ok := vt.prevotes[addrStr]; ok {
            status.HasPrevoted = true
            status.PrevoteBlockHash = vote.BlockID.Hash
        }

        if vote, ok := vt.precommits[addrStr]; ok {
            status.HasPrecommitted = true
            status.PrecommitBlockHash = vote.BlockID.Hash
        }

        result = append(result, status)
    }

    return result
}

type ValidatorVoteStatus struct {
    Address            types.Address `json:"address"`
    VotingPower        int64         `json:"votingPower"`
    PowerPercent       float64       `json:"powerPercent"`
    HasPrevoted        bool          `json:"hasPrevoted"`
    PrevoteBlockHash   []byte        `json:"prevoteBlockHash,omitempty"`
    HasPrecommitted    bool          `json:"hasPrecommitted"`
    PrecommitBlockHash []byte        `json:"precommitBlockHash,omitempty"`
}
```

3.4 메트릭 수집

```go
// tools/consensus-viz/metrics.go
package consensusviz

import (
    "time"

    "github.com/prometheus/client_golang/prometheus"
    "github.com/prometheus/client_golang/prometheus/promauto"
)

var (
    roundDuration = promauto.NewHistogramVec(
        prometheus.HistogramOpts{
            Name:    "consensus_round_duration_seconds",
            Help:    "Duration of consensus rounds",
            Buckets: []float64{0.5, 1, 2, 5, 10, 30, 60},
        },
        []string{"result"}, // "committed" or "timeout"
    )

    voteLatency = promauto.NewHistogramVec(
        prometheus.HistogramOpts{
            Name:    "consensus_vote_latency_seconds",
            Help:    "Time from round start to vote received",
            Buckets: []float64{0.1, 0.25, 0.5, 1, 2, 5},
        },
        []string{"vote_type", "validator"},
    )

    quorumTime = promauto.NewHistogramVec(
        prometheus.HistogramOpts{
            Name:    "consensus_quorum_time_seconds",
            Help:    "Time to reach quorum (2/3+)",
            Buckets: []float64{0.5, 1, 2, 5, 10},
        },
        []string{"vote_type"}, // "prevote" or "precommit"
    )

    missedVotes = promauto.NewCounterVec(
        prometheus.CounterOpts{
            Name: "consensus_missed_votes_total",
            Help: "Total number of missed votes by validator",
        },
        []string{"validator", "vote_type"},
    )

    roundsPerBlock = promauto.NewHistogram(
        prometheus.HistogramOpts{
            Name:    "consensus_rounds_per_block",
            Help:    "Number of rounds before block commit",
            Buckets: []float64{1, 2, 3, 5, 10},
        },
    )
)

// MetricsCollector collects consensus metrics
type MetricsCollector struct {
    roundStartTime time.Time
    prevoteStart   time.Time
    precommitStart time.Time
}

func (mc *MetricsCollector) OnRoundStart(height int64, round int32) {
    mc.roundStartTime = time.Now()
    mc.prevoteStart = time.Time{}
    mc.precommitStart = time.Time{}
}

func (mc *MetricsCollector) OnVote(vote *Vote, validator string) {
    latency := time.Since(mc.roundStartTime).Seconds()

    switch vote.Type {
    case PrevoteType:
        voteLatency.WithLabelValues("prevote", validator).Observe(latency)
    case PrecommitType:
        voteLatency.WithLabelValues("precommit", validator).Observe(latency)
```

```go
    }
}

func (mc *MetricsCollector) OnQuorumReached(voteType string) {
    duration := time.Since(mc.roundStartTime).Seconds()
    quorumTime.WithLabelValues(voteType).Observe(duration)
}

func (mc *MetricsCollector) OnRoundEnd(result string, round int32) {
    duration := time.Since(mc.roundStartTime).Seconds()
    roundDuration.WithLabelValues(result).Observe(duration)

    if result == "committed" {
        roundsPerBlock.Observe(float64(round + 1))
    }
}

func (mc *MetricsCollector) OnMissedVote(validator, voteType string) {
    missedVotes.WithLabelValues(validator, voteType).Inc()
}
```

---

4. Performance Profiler 도구

4.1 개요

EVM 및 Cosmos SDK 레이어의 성능을 자동으로 분석하는 도구.

4.2 자동 병목 탐지

```go
// tools/profiler/bottleneck.go
package profiler

import (
    "runtime/pprof"
    "sort"
    "strings"
)

// BottleneckDetector automatically identifies performance bottlenecks
type BottleneckDetector struct {
    thresholds BottleneckThresholds
}

type BottleneckThresholds struct {
    CPUHotspotPercent   float64 // Function taking > X% of CPU
    AllocHotspotBytes   int64   // Function allocating > X bytes
    GoroutineBlockMs    int64   // Goroutine blocked > X ms
    MutexContention     float64 // Mutex contention > X%
}

var DefaultThresholds = BottleneckThresholds{
    CPUHotspotPercent: 5.0,
    AllocHotspotBytes: 1024 * 1024, // 1MB
    GoroutineBlockMs:  100,
    MutexContention:   1.0,
}

type Bottleneck struct {
    Type        BottleneckType `json:"type"`
    Location    string         `json:"location"`
    Description string         `json:"description"`
    Impact      float64        `json:"impact"`
    Suggestion  string         `json:"suggestion"`
}

type BottleneckType string

const (
    BottleneckCPU         BottleneckType = "cpu"
    BottleneckMemory      BottleneckType = "memory"
    BottleneckGoroutine   BottleneckType = "goroutine"
    BottleneckMutex       BottleneckType = "mutex"
    BottleneckIO          BottleneckType = "io"
)

// Analyze performs bottleneck analysis
func (bd *BottleneckDetector) Analyze(profile *pprof.Profile) []Bottleneck {
    var bottlenecks []Bottleneck

    // Analyze CPU hotspots
    cpuBottlenecks := bd.analyzeCPU(profile)
    bottlenecks = append(bottlenecks, cpuBottlenecks...)

    // Analyze memory allocations
    memBottlenecks := bd.analyzeMemory(profile)
    bottlenecks = append(bottlenecks, memBottlenecks...)

    // Sort by impact
    sort.Slice(bottlenecks, func(i, j int) bool {
        return bottlenecks[i].Impact > bottlenecks[j].Impact
    })

    return bottlenecks
}

func (bd *BottleneckDetector) analyzeCPU(profile *pprof.Profile) []Bottleneck {
    var bottlenecks []Bottleneck

    // Known EVM-related bottlenecks
    evmPatterns := map[string]string{
        "statedb.GetState":     "Storage read - consider caching",
        "keeper.SetState":      "Storage write - consider batching",
        "journal.append":       "Journal expansion - consider pre-allocation",
        "crypto.Keccak256":     "Hashing - minimize hash operations",
        "rlp.Decode":           "RLP decoding - cache decoded results",
```

```go
        "core.NewStateDB":      "StateDB creation - reuse where possible",
    }

    for location, suggestion := range evmPatterns {
        // Check if pattern appears in profile hotspots
        if samples, found := findFunction(profile, location); found {
            percent := float64(samples) / float64(profile.TotalSamples) * 100
            if percent > bd.thresholds.CPUHotspotPercent {
                bottlenecks = append(bottlenecks, Bottleneck{
                    Type:        BottleneckCPU,
                    Location:    location,
                    Description: fmt.Sprintf("%.1f%% of CPU time", percent),
                    Impact:      percent,
                    Suggestion:  suggestion,
                })
            }
        }
    }

    return bottlenecks
}

// GenerateReport creates a human-readable report
func (bd *BottleneckDetector) GenerateReport(bottlenecks []Bottleneck) string {
    var sb strings.Builder

    sb.WriteString("Performance Bottleneck Report\n")
    sb.WriteString("============================\n\n")

    if len(bottlenecks) == 0 {
        sb.WriteString("No significant bottlenecks detected.\n")
        return sb.String()
    }

    for i, b := range bottlenecks {
        sb.WriteString(fmt.Sprintf("%d. [%s] %s\n", i+1, b.Type, b.Location))
        sb.WriteString(fmt.Sprintf("   Impact: %.1f%%\n", b.Impact))
        sb.WriteString(fmt.Sprintf("   Description: %s\n", b.Description))
        sb.WriteString(fmt.Sprintf("   Suggestion: %s\n\n", b.Suggestion))
    }

    return sb.String()
}
```

4.3 자동 프로파일링 스케줄러

```go
// tools/profiler/scheduler.go
package profiler

import (
    "context"
    "os"
    "runtime/pprof"
    "sync"
    "time"
)

// ProfileScheduler automatically collects profiles based on conditions
type ProfileScheduler struct {
    mu          sync.Mutex
    config      SchedulerConfig
    detector    *BottleneckDetector

    // Profile storage
    profiles    []*StoredProfile
    maxProfiles int
}

type SchedulerConfig struct {
    // Periodic profiling
    CPUInterval   time.Duration
    HeapInterval  time.Duration

    // Triggered profiling
    TxThreshold   int     // Profile if TPS drops below this
    LatencyMs     int64   // Profile if latency exceeds this
    MemoryMB      int64   // Profile if memory exceeds this

    // Output
    OutputDir     string
}

type StoredProfile struct {
    Type      string    `json:"type"`
    Timestamp time.Time `json:"timestamp"`
    Trigger   string    `json:"trigger"`
    FilePath  string    `json:"filePath"`
    Analysis  []Bottleneck `json:"analysis,omitempty"`
}

func (ps *ProfileScheduler) Start(ctx context.Context) {
    // Periodic CPU profiling
    go ps.periodicCPUProfile(ctx)

    // Periodic heap profiling
    go ps.periodicHeapProfile(ctx)

    // Condition-based profiling
    go ps.monitorConditions(ctx)
}

func (ps *ProfileScheduler) periodicCPUProfile(ctx context.Context) {
    ticker := time.NewTicker(ps.config.CPUInterval)
    defer ticker.Stop()

    for {
        select {
        case <-ctx.Done():
            return
        case <-ticker.C:
            ps.captureCPUProfile("periodic")
        }
    }
}

func (ps *ProfileScheduler) captureCPUProfile(trigger string) error {
    filename := fmt.Sprintf("%s/cpu_%s_%s.pprof",
        ps.config.OutputDir,
        time.Now().Format("20060102_150405"),
        trigger)

    f, err := os.Create(filename)
```

```go
    if err != nil {
        return err
    }
    defer f.Close()

    if err := pprof.StartCPUProfile(f); err != nil {
        return err
    }

    time.Sleep(30 * time.Second)
    pprof.StopCPUProfile()

    // Analyze profile
    profile, err := pprof.Parse(f)
    if err == nil {
        analysis := ps.detector.Analyze(profile)
        ps.storeProfile(&StoredProfile{
            Type:      "cpu",
            Timestamp: time.Now(),
            Trigger:   trigger,
            FilePath:  filename,
            Analysis:  analysis,
        })
    }

    return nil
}

func (ps *ProfileScheduler) monitorConditions(ctx context.Context) {
    ticker := time.NewTicker(5 * time.Second)
    defer ticker.Stop()

    for {
        select {
        case <-ctx.Done():
            return
        case <-ticker.C:
            var m runtime.MemStats
            runtime.ReadMemStats(&m)

            // Check memory threshold
            memMB := int64(m.Alloc / 1024 / 1024)
            if memMB > ps.config.MemoryMB {
                ps.captureHeapProfile("high_memory")
            }
        }
    }
}
```

5. CLI 통합

5.1 통합 CLI 도구

```go
// tools/evmtools/main.go
package main

import (
    "github.com/spf13/cobra"
)

func main() {
    rootCmd := &cobra.Command{
        Use:   "evmtools",
        Short: "Cosmos EVM Developer Tools",
    }

    // State Inspector
    inspectCmd := &cobra.Command{
        Use:   "inspect",
        Short: "EVM State Inspector",
    }
    inspectCmd.AddCommand(
        newInspectStateCmd(),
        newInspectDiffCmd(),
        newInspectWatchCmd(),
    )

    // Consensus Visualizer
    consensusCmd := &cobra.Command{
        Use:   "consensus",
        Short: "Consensus Visualizer",
    }
    consensusCmd.AddCommand(
        newConsensusWatchCmd(),
        newConsensusHistoryCmd(),
        newConsensusMetricsCmd(),
    )

    // Profiler
    profileCmd := &cobra.Command{
        Use:   "profile",
        Short: "Performance Profiler",
    }
    profileCmd.AddCommand(
        newProfileCPUCmd(),
        newProfileMemCmd(),
        newProfileAnalyzeCmd(),
    )

    // Tracer
    traceCmd := &cobra.Command{
        Use:   "trace",
        Short: "Transaction Tracer",
    }
    traceCmd.AddCommand(
        newTraceTxCmd(),
        newTraceBlockCmd(),
        newTraceCallCmd(),
    )

    rootCmd.AddCommand(inspectCmd, consensusCmd, profileCmd, traceCmd)

    if err := rootCmd.Execute(); err != nil {
        os.Exit(1)
    }
}
```

5.2 사용 예시

```
# State Inspector
evmtools inspect state --address 0x1234...5678
evmtools inspect diff --from-block 100 --to-block 200
evmtools inspect watch --filter "type=storage_change,account=0x..."

# Consensus Visualizer
evmtools consensus watch --height latest
evmtools consensus history --blocks 100
evmtools consensus metrics --prometheus :9090

# Profiler
evmtools profile cpu --duration 30s --output cpu.pprof
evmtools profile mem --output heap.pprof
evmtools profile analyze --input cpu.pprof --report bottlenecks.txt

# Tracer
evmtools trace tx 0xabcd...ef01 --tracer call
evmtools trace block 12345 --format json
evmtools trace call --to 0x1234 --data 0x70a08231... --tracer prestate
```

---

6. Web UI 설계

6.1 대시보드 구조

```
+-------------------------------------------------------------+
|  Cosmos EVM Developer Dashboard                        [Dark] |
+-------------------------------------------------------------+
| [State] [Consensus] [Profiler] [Traces] [Logs]               |
+-------------------------------------------------------------+
|                                                              |
|  +------------------------+  +----------------------------+  |
|  |   Block Info           |  |   Performance Metrics       |  |
|  |   Height: 12345678     |  |   TPS: 1,234               |  |
|  |   Txs: 156             |  |   Block Time: 2.1s          |  |
|  |   Gas Used: 12.5M      |  |   Memory: 4.2 GB           |  |
|  +------------------------+  +----------------------------+  |
|                                                              |
|  +---------------------------------------------------------+ |
|  |   Recent State Changes                                   | |
|  +---------------------------------------------------------+ |
|  | Time     | Account        | Type    | Details            | |
|  |----------|----------------|---------|--------------------| |
|  | 14:30:45 | 0x1234...5678 | Balance | -1.2 ETH           | |
|  | 14:30:44 | 0xabcd...ef01 | Storage | slot[0x01] updated | |
|  +---------------------------------------------------------+ |
|                                                              |
+-------------------------------------------------------------+
```

6.2 React 컴포넌트 구조

```
// web/src/components/Dashboard.tsx
import React from 'react';
import { useWebSocket } from '../hooks/useWebSocket';

export const Dashboard: React.FC = () => {
    const { events, connected } = useWebSocket('ws://localhost:8080/ws');

    return (
        <div className="dashboard">
            <header>
                <h1>Cosmos EVM Developer Dashboard</h1>
                <StatusIndicator connected={connected} />
            </header>

            <nav>
                <NavLink to="/state">State</NavLink>
                <NavLink to="/consensus">Consensus</NavLink>
                <NavLink to="/profiler">Profiler</NavLink>
                <NavLink to="/traces">Traces</NavLink>
            </nav>

            <main>
                <Routes>
                    <Route path="/state" element={<StateInspector />} />
                    <Route path="/consensus" element={<ConsensusViz />} />
                    <Route path="/profiler" element={<Profiler />} />
                    <Route path="/traces" element={<TraceViewer />} />
                </Routes>
            </main>
        </div>
    );
};

// State Inspector Component
const StateInspector: React.FC = () => {
    const [filter, setFilter] = useState<EventFilter>({});
    const { events } = useStateEvents(filter);

    return (
        <div className="state-inspector">
            <FilterPanel filter={filter} onChange={setFilter} />
            <EventList events={events} />
            <StateDiffViewer />
        </div>
    );
};

// Consensus Visualizer Component
const ConsensusViz: React.FC = () => {
    const { state, validators, votes } = useConsensusState();

    return (
        <div className="consensus-viz">
            <RoundInfo height={state.height} round={state.round} step={state.step} />
            <ValidatorVotesTable validators={validators} votes={votes} />
            <QuorumProgress prevotes={votes.prevotes} precommits={votes.precommits} />
            <RoundTimeline history={state.history} />
        </div>
    );
};
```

7. 구현 로드맵

7.1 Phase 1: 핵심 기능 (Week 1-2)

1. **State Inspector 기본**
    - StateDB 트레이서 훅 구현
    - 이벤트 버퍼 및 구독 시스템
    - CLI 기본 명령어
2. **Consensus Visualizer 기본**
    - 투표 추적 로직
    - 메트릭 수집
    - CLI 기본 명령어

7.2 Phase 2: 고급 기능 (Week 3-4)

1. **WebSocket 스트리밍**
    - 실시간 이벤트 스트리밍
    - 필터링 시스템
2. **Performance Profiler**
    - 자동 병목 탐지
    - 프로파일 스케줄러

7.3 Phase 3: UI 및 통합 (Week 5-6)

1. **Web UI**
    - React 대시보드
    - 시각화 컴포넌트
2. **통합 및 문서화**
    - CLI 도구 통합
    - 사용자 문서

---

8. 결론

8.1 도구 요약

도구	핵심 기능	사용 사례
State Inspector	실시간 상태 모니터링, Diff 시각화	디버깅, 감사
Consensus Visualizer	투표 추적, 메트릭 수집	네트워크 모니터링
Performance Profiler	자동 병목 탐지, 리포트 생성	성능 최적화

8.2 기대 효과

1. **개발 생산성 향상**: 디버깅 시간 50% 단축
2. **운영 효율성**: 실시간 모니터링으로 장애 조기 탐지
3. **성능 최적화**: 자동화된 병목 분석

8.3 참고 코드 위치

- 트레이서: x/vm/types/tracer.go
- Debug API: rpc/namespaces/ethereum/debug/api.go
- Tracing Backend: rpc/backend/tracing.go

문서 버전: 1.0 **작성일**: 2025-01 **작성자**: B-Harvest Core Developer Candidate **페이지 수**: ~25 페이지